

Experiment Title 8: Covariance and Contravariance in Java Generics

Problem Statement: You are designing an employee management system where different types of employees (Managers and Developers) perform distinct tasks. The system should:

- Use inheritance to define common and specialized behaviors
- Use generic wildcards to safely read from and write to lists of employees
- Demonstrate covariance (<? extends Employee>) for reading
- Demonstrate contravariance (<? super Developer>) for writing

Experiment Task

1. Implement the given program structure.
2. Observe how printEmployees() uses covariance to read from lists of Manager and Developer.
3. Observe how addDevelopers() uses contravariance to add Developer objects to a general list.
4. Modify the program to add more employee types and test wildcard behavior.

Sample Input

No user input is required. The program runs autonomously.

Sample Output

Employee Report:

Charlie is writing code

Employee Report:

Diana is managing team

Developers added successfully!

Employee Report:

Alice is writing code

Bob is writing code

Note: Output may vary based on added employee names or types.

Experiment 9: Class Inspection Using Reflexive Programming

Objective: To understand and implement reflexive programming in Java by inspecting class metadata using the Reflection API. Learners will dynamically access class names, fields, methods, and constructors at runtime, enabling flexible design, introspection, and metadata-driven programming.

Reflection can be used to get information about class, constructors, and methods as depicted below in tabular format as shown.

Class	The getClass() method is used to get the name of the class to which an object belongs.
Constructors	The getConstructors() method is used to get the public constructors of the class to which an object belongs.
Methods	The getMethods() method is used to get the public methods of the class to which an object belongs.

We can invoke a method through reflection if we know its name and parameter types.

We use two methods for this purpose as described below before moving ahead as follows:

1. getDeclaredMethod()
2. invoke()

Method 1: getDeclaredMethod(): It creates an object of the method to be invoked.

Syntax: The syntax for this method

Class.getDeclaredMethod(name, parametertype)

Parameters:

- Name of a method whose object is to be created and An array of Class objects

Method 2: invoke(): It invokes a method of the class at runtime we use the following method.

Syntax:

Method.invoke(Object, parameter)

Note: If the method of the class doesn't accept any parameter then null is passed as an argument.

Important Reflection Methods

Reflection Method	Purpose
Class.forName()	Loads the class dynamically
getDeclaredFields()	Retrieves all declared fields
getDeclaredMethods()	Retrieves all declared methods
getDeclaredConstructors()	Retrieves all declared constructors
getName()	Gets the name of a field/method
getModifiers()	Gets access modifiers
getParameterTypes()	Gets constructor/method parameter types

9.1: Implement Dynamic Class Inspection Using Java Reflection API

Problem Statement Scenario: You are developing a debugging tool that must display the internal details of any Java class without knowing its structure at compile time. The tool should dynamically inspect the class and display its name, fields, methods, and constructors.

Develop a Java program to implement a class-inspection tool using the Java Reflection API. The program should accept a class name at runtime and dynamically load the class using `Class.forName()`. Without directly accessing the class members, display the class name, all declared fields, all declared methods, and all declared constructors, including constructor parameter types and access modifiers. The program should work for any valid Java class and handle the exception when the specified class cannot be found.

Requirements: The program should:

1. Accept the class name as input.
2. Load the class dynamically using `Class.forName()`.
3. Display the class name.
4. Retrieve and display all fields using `getDeclaredFields()`.
5. Retrieve and display all methods using `getDeclaredMethods()`.
6. Retrieve and display all constructors using `getDeclaredConstructors()`.
7. Display the complete constructor signature including access modifier and parameter types.
8. Handle the exception if the specified class does not exist.

Sample Input:

Enter class name: Student

Sample Output:

Class Name: Student

Fields:

- name
- marks

Methods:

- display
- secretNote

Constructors:

- public Student(java.lang.String,int)
- public Student()

Experiment Title 9.2: Implement Dynamic Class Loading, Object Creation and Method Invocation Using Java Reflection (Dynamic Instantiation Using Java Reflection)

Problem Statement: A software application needs to create objects **at runtime** without knowing the exact class at compile time. The class name will be provided as a String during program execution.

Consider the following classes:

```
class Student {  
    public void display() {  
        System.out.println("Hello, I am a Student!");  
    }  
}  
class Teacher {  
    public void display() {  
        System.out.println("Hello, I am a Teacher!");  
    }  
}
```

Develop a Java program using the Reflection API to dynamically:

1. Accept a class name such as "Student" or "Teacher" as a String.
2. Load the specified class using Class.forName().
3. Create an object of the loaded class dynamically using reflection.
4. Locate the display() method using getMethod().
5. Invoke the display() method using invoke().
6. Handle possible exceptions appropriately.

Requirements

The program should not use:
new Student(); and new Teacher();

Instead, the class must be loaded and instantiated completely at runtime.

Expected Output

For input: Teacher

Output:

Class loaded: Teacher
Object created successfully.
Hello, I am a Teacher!
Method invoked successfully.

For input: Student

Output:

Class loaded: Student
Object created successfully.
Hello, I am a Student!
Method invoked successfully.